

DealLens — Project Overview

A plain-English guide to what this is, how it works, and what's real vs. illustrative. Read this first if you're picking the project up.

1. What it is

DealLens is a **due-diligence and business-valuation platform**. You give it a company's financials (type them, upload a statement, or import from accounting software), tell it the business type, and it returns a **defensible value range** with the reasoning shown — plus a diligence checklist, risk flags, and a client-ready report. It runs as a **multi-user web app** and is **deployed live** on Railway.

It was built to answer a simple question — “*what is a business worth, and should I buy it?*” — with the discipline a professional advisor would use, made accessible to non-experts.

2. What it can value (and with the *right* method)

Different businesses are valued differently. DealLens has three distinct paths:

Business type	Method	In the app
Operating companies (a bakery → a large public firm)	Normalize earnings → DCF + market multiples + net asset value → triangulate; diligence risk adjusts it	New deal / ✗ Quick screen
Banks & financial institutions	Price-to-book (P/B) and price-to-earnings (P/E) — not EBITDA, because interest is a bank's revenue	Bank
Conglomerates (Amazon-type multi-business)	Sum-of-the-parts: value each segment on its own comp, then combine	Conglomerate

For operating companies there's also an **SMB vs. Public** tier toggle — small private businesses trade at low multiples (2–5×), large public ones much higher (10×+). Picking the right tier is what made a public company like Saia value near its real market cap instead of a fraction of it.

What it is *not* built for (would need new methods): pre-revenue startups (valued on potential, not earnings), and anything requiring specialist models beyond the three above.

3. How it's built (in plain terms)

DealLens is **13 small, independent “primitives”** — each does one job, is separately tested, and talks through one simple contract (`invoke(payload) → {ok, result}`). A **gateway** exposes them over the web, and a **single-page UI** sits on top. This is why it was safe to extend so fast: adding “banking” or a “public tier” didn't touch the rest.

#	Primitive	Job
1	valuation_engine	The core math: normalization, DCF, multiples, NAV, triangulation, sensitivity
2	diligence_engine	Checklist templates, risk roll-up, red-flag detection
3	comparables_library	Sector multiple bands (SMB + public tiers) with size/growth modifiers
4	orchestrator	One call runs diligence → comparables → valuation
5	report	Renders a result into HTML / Markdown / Word
6	workspace	Persistent deals: create/list/evaluate/report, sharing, comments
7	quickscreen	Fast indicative valuation + asking-price verdict
8	documents	Extract financials

		from CSV / Excel / Word / PDF / pasted text
9	assist	Suggest add-backs, draft the valuation narrative (rule-based)
10	accounts	User accounts + sessions (password hashing)
11	integrations	QuickBooks / Xero OAuth + import
12	banking	Financial-institution valuation (P/B, P/E)
13	sotp	Sum-of-the-parts for conglomerates

Plus deallens_ui (the web app + gateway) and deallens_integration (a fuzz test that throws thousands of random deals at the whole system).

Status: 228 automated tests passing. Two deliberate hardening passes fixed real bugs (loss-making/insolvent inputs, and silent number-extraction errors like thousands/millions scaling), and the fuzz suite confirms the assembled system holds under hostile inputs.

4. How to use it

Live app: <https://deallens-production-7e86.up.railway.app> (sign up, then):

- **+ New deal** — operating company. Type financials or **upload a statement** (Excel/CSV/Word/PDF) to auto-fill, pick sector + metric + tier, **Create & evaluate**. Share with teammates, add comments, generate a report.
- **⚡ Quick screen** — a fast ballpark; add an asking price for a fair/expensive verdict.
- **Bank** — institution type + net income + book value → P/B & P/E valuation.
- **Conglomerate** — add segments (each with its own sector/earnings) → summed value.

Everything is also callable via the API (/api/...) and each primitive has a CLI.

5. How it's deployed

- One **Docker** container, pure Python standard library (no heavy dependencies).
- Hosted on **Railway**, auto-deploys on every git push to main.

- Data in **SQLite** on a mounted volume (DEALLENB_DB=/data/deallens.db).
- Key env vars: DEALLENB_DB, DEALLENB_HOME, DEALLENB_SECRET, and (for accounting imports) QB_*, XERO_*, DEALLENB_BASE_URL.
- Full steps in DEPLOY.md; accounting setup in INTEGRATIONS_SETUP.md.

To move beyond SQLite (many concurrent users), swap the store for Postgres — the persistence interface is already abstracted; it's a new Store class + a connection string, no engine changes.

6. What's REAL vs. ILLUSTRATIVE (read this before trusting a number)

This is the most important section. The **methods and math are real and tested**. Several **inputs are illustrative defaults** you should replace before relying on outputs for an actual decision:

Component	Status
DCF, normalization, triangulation, P/B–P/E, sum-of-the-parts math	Real — implemented to standard methodology and unit-tested
Number extraction, risk roll-up, report generation, auth	Real — working and hardened
Comparable multiple bands (SMB, public, bank P/B, holdco discount)	Illustrative — sensible defaults, <i>not</i> a live market feed. Every report says so. Replace with vetted comps.
Diligence checklist templates	Real structure, generic content — good starting checklists; tailor to the deal
AI assist (add-back suggestions, narrative)	Real but deterministic/rule-based, not an LLM — a clean seam exists to add a real model later
QuickBooks/Xero import	Real OAuth flow, mock provider by default — needs your developer credentials to go live

Bottom line: **DealLens computes correctly from whatever comps and assumptions you give it**. Out of the box those are tuned illustratively; the single highest-value next step is loading multiples you trust. It is **decision-support, not advice** — it exposes every assumption so a human can judge it, and it never transacts.

7. How to extend it (the pattern)

Every addition so far followed the same shape, so the next one can too:

- **New multiples / a new tier** → edit `deallens_comparables_library/dataset.py` (this is how the “public” tier was added — pure data).
 - **A new valuation method** (e.g. a REIT or insurance model) → add a new primitive with an `invoke()` + tests, then a gateway route + a UI form (this is how “banking” and “sum-of-the-parts” were added).
 - **A real LLM for assist** → swap the rule tables in `deallens_assist` behind the same interface; add an API key.
-

8. Where things live

Financial Analysis/

README.md	# suite map + how to run everything
OVERVIEW.md	# this file
ROADMAP.md	# what's built vs. what's next
DEPLOY.md	# Railway deployment steps
INTEGRATIONS_SETUP.md	# QuickBooks / Xero setup
DealLens_... PRD.docx	# original concept/architecture/design/PRD
Dockerfile, railway.toml	# deployment config
deallens_*/	# the 13 primitives + UI (+ integration tests)
README.md, START_HERE.md	# per-primitive docs

Run the whole test suite: see the loop in README.md (228 tests).

9. Honest boundaries

- Multiples are illustrative until you replace them (see §6).
- It values operating companies, banks, and conglomerates; other structures (startups, project finance, etc.) would need new methods.
- The live app is a research/preview deployment — before wide public use, add rate-limiting on auth, email verification/password reset, and consider Postgres
 - encrypted token storage.
- Decision-support only: not financial, legal, or valuation advice; it never moves money or executes trades.